JC2001: Introduction to Software Engineering

Lecture 5: Requirements engineering

Lecture Schedule: Week 4, Day 1

Dr Zhenhua Yu zhenhua.yu@abdn.ac.uk

Dr Riazul Islam riazul.islam@abdn.ac.uk

Dr Shahzad Mumtaz shahzad.mumtaz@abdn.ac.uk

Course Delivery Team

Dr Zhenhua Yu

zhenhua.yu@abdn.ac.uk



Dr Riazul Islam

riazul.islam@abdn.ac.uk



Dr Shahzad Mumtaz

shahzad.mumtaz@abdn.ac.uk



Learning objectives

- Understand the concepts of user and system requirements.
- Understand the differences between functional and non-functional requirements.
- Understand the main requirements engineering activities of elicitation, analysis, and validation, and the relationship between these activities.
- Understand why requirements engineering is necessary.

Reference: Ian Sommerville, Software Engineering – Chapter 4, 10th edition, Pearson, 2015.

Outline of lecture

- Introduce user and system requirements.
- Describe functional and non-functional requirements.
- Describe the requirements engineering process:
    - Requirements elicitation.
    - Requirements specification.
    - Requirements validation.
    - Requirements change.

Definition of requirements engineering

- The requirements for a system are the descriptions of the **services** that a system should provide and the **constraints** on its operation.
- These requirements reflect the **needs of customers** for a system that serves a certain purpose such as controlling a device, placing an order, or finding information.
- The **process** of finding out, analysing, documenting and checking these services and constraints is called *requirements engineering* (RE).

User requirements

- User requirements is defined as **high-level statements** that are written in a natural language, plus diagrams, of what services the system is expected to provide to system users and the constraints under which it must operate.
- The user requirements may vary from *broad statements of the system features* required to *detailed, precise descriptions of the system functionality*.

Example of a user requirement

The Mentcare system shall generate monthly management reports showing the cost of drugs prescribed by each clinic during that month.

System requirements

- System requirements are more **detailed** descriptions of the software system's functions, services, and operational constraints.
- The system requirements document (sometimes called a functional specification) should define exactly *what is to be implemented*. It may be part of a contract between the system buyer and the software developers.

Example of a system requirement

1. On the last working day of each month, a summary of the drugs prescribed, their cost and the prescribing clinics shall be generated.
2. The system shall generate the report for printing after 17.30 on the last working day of the month.

Mock quiz

- What are the differences between a user requirement and a system requirement?

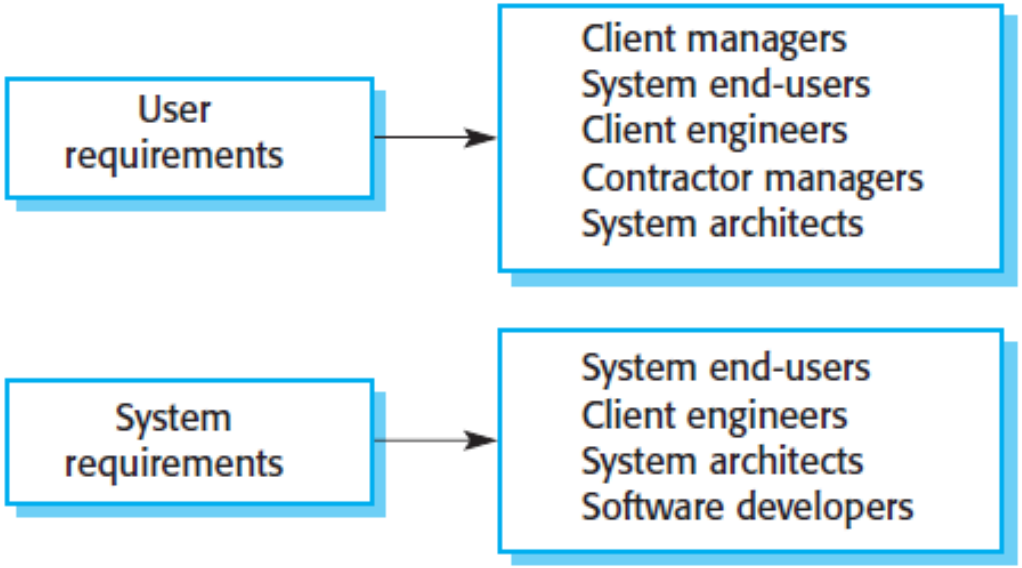
A. A user requirement provides detailed description of what a system should do. A system requirement gives more details on the implementation.

B. A user requirement is a high-level statement of what services the system is expected to provide. A system requirement is a more detailed description of services provided by a system and specifies exactly what is to be implemented.

C. A user requirement describes a system function. A system requirement describes the function's implementation details.

User and system requirements

- The user and system requirements should be written at **different levels of detail** because **different types of readers** use them in different ways.



System stakeholders

- The system stakeholders include anyone who is affected by the system in some way and so anyone who has a **legitimate interest** in it.
- Stakeholders range from **end-users** of a system through **managers** to external stakeholders such as **regulators**.
- For example, the stakeholders of the Mentcare system include:
  - **Patients.** Their information is recorded in the system.
  - **Doctors.** They are responsible for assessing and treating patients.
  - **Nurses.** They coordinate the consultations with doctors and administer some treatments.
  - **IT staff.** They are responsible for installing and maintaining the system.
  - **Medical ethics manager.** They ensure that the system meets current ethical guidelines for patient care.

Functional requirements

- These are **statements of services** the system should provide, how the system should react to particular inputs, and how the system should behave in particular situations.
- In some cases, the functional requirements may also explicitly state what the system should do.
- When expressed as **user requirements**, functional requirements should be written in natural language so that **system users and managers can understand them**.

Functional requirements: Mentcare system

- Examples of functional requirements for the Mentcare system, used to maintain information about patients receiving treatment for mental health problems.
    - **Function 1**: A user shall be able to search the appointments lists for all clinics.
    - **Function 2**: The system shall generate each day, for each clinic, a list of patients who are expected to attend appointments that day.
    - **Function 3**: Each staff member using the system shall be uniquely identified by their eight-digit employee number.

- The above user requirements define **specific functionality** that should be included in the system. Contrast functions 1 and 3, functional requirements can be written at different levels of detail.

Functional requirements: ATM machine

- The mandatory actions an ATM machine should perform upon user request:
  - **Function 1**: Input the selection to withdraw funds, the ATM *should* dispense the selected amount.
  - **Function 2**: Input the selection to receive an account balance, the ATM *should* display the available amount.
  - **Function 3**: Indicates that all transactions have ended, the ATM *should* return the card and return to its idle state.

Functional requirements: ATM machine

- The above-mentioned requirements are not unconditional. The ATM can only respond accordingly under the **appropriate conditions**:
  - **Condition 1**: The ATM *should* verify the card and PIN data entered into the ATM.
  - **Condition 2**: The ATM *should* display the operations performed by the withdrawal, balance enquiry, deposit and transfer screens only when the card and PIN data is verified by the bank.
  - **Condition 3**: The ATM *should* dispense the selected amount only when there is sufficient balance in the account.

Non-functional requirements

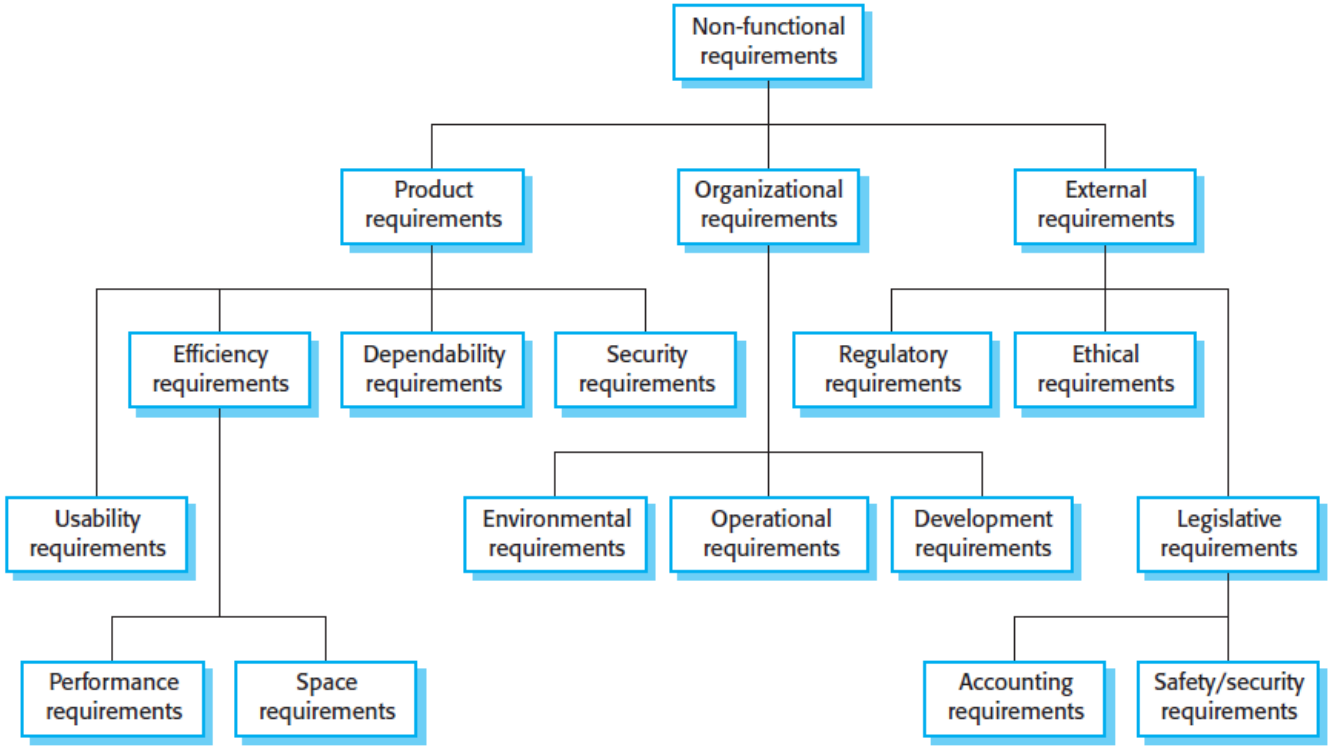
- Non-functional requirements usually **specify constrains** on the characteristics of the system as a whole. They may relate to emergent system properties such as reliability, response time, and memory use.

- Non-functional requirements are often more critical than individual functional requirements.
    - The reason is failing to meet a non-functional requirement can mean that the whole system is unusable.
    - For example, if the ATM machine does not meet the security requirements, it will not be certified as secure for operation.

Mock quiz

- Why are non-functional requirements more critical than functional requirements?
    A. Failing to meet a non-functional requirement means that the whole system becomes unusable.
    B. Non-functional requirements help in securing a system from failures.
    C. Non-functional requirements are better than functional requirements.
    D. Non-functional requirements specify constrains on system performance.

Types of non-functional requirements



Product requirements

- These requirements specify or constrain the **runtime behaviour** of the system.

Examples include:

- Performance requirements for how fast the system must execute and how much memory it requires.
- Reliability requirements that set out the acceptable failure rate.
- Security requirements.
- Usability requirements.

Organisational requirements

- These requirements are broad system requirements derived from **policies and procedures** in the customer's and developer's organisations. Examples include:
    - Operational process requirements that define how the system will be used.
    - Development process requirements that specify the programming language.
    - The development environment or process standards to be used.
    - The environmental requirements that specify the operating environment of the system.

External requirements

- This broad heading covers all requirements that are derived from **factors external** to the system and its development process. These may include:
    - Regulatory requirements that set out what must be done for the system to be approved for use by a regulator.
    - Legislative requirements that must be followed to ensure that the system operates within the law.
    - Ethical requirements that ensure that the system will be acceptable to its users and the general public.

Non-functional requirements: Mentcare

- Product requirement:
  - The Mentcare system shall be available to all clinics during normal working hours (Monday to Friday, 08.30 to 17.30). Downtime within normal working hours shall not exceed 5 seconds in any one day.
- Organisational requirement:
  - Users of the Mentcare system shall identify themselves using their health authority identity card.
- External requirement:
  - The system shall implement patient privacy provisions as set out in the General Data Protection Regulation (GDPR).

Non-functional requirements: ATM

- Product requirement:
  - The ATM shall be available 24 hours per day. Downtime shall not exceed 2 hours per week.
  - The cash dispense can be opened and refilled with cash.
- External requirement:
  - The ATM will service one customer at a time.
  - The PIN number must be entered correctly within three attempts.

Mock quiz

- You are the company's lead developer responsible for managing a suite a programming languages. A new software project requires you to use a new programming language. However, your team is not familiar with the new programming language, so you need to consider the pros and cons in integrating a new programming language into your software development framework. What are the factors involved in reaching a decision? Discuss your answer.

Requirements engineering processes

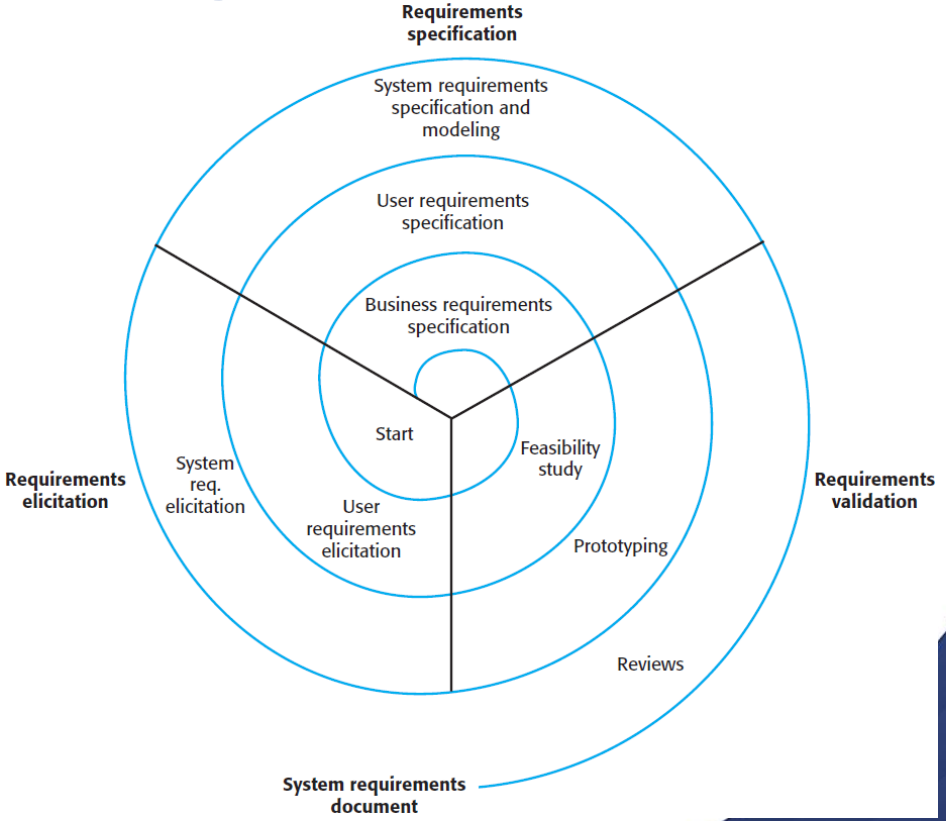
- Requirements engineering involves three key activities:
  - **Requirements elicitation**, which involves discovering requirements by interacting with stakeholders.
  - **Requirements specification**, which involves converting these requirements into a standard form.
  - **Requirements validation**, which involves checking that the requirements actually define the system that the customer wants.

- Requirements engineering is an iterative process in which the above-mentioned activities are interleaved.

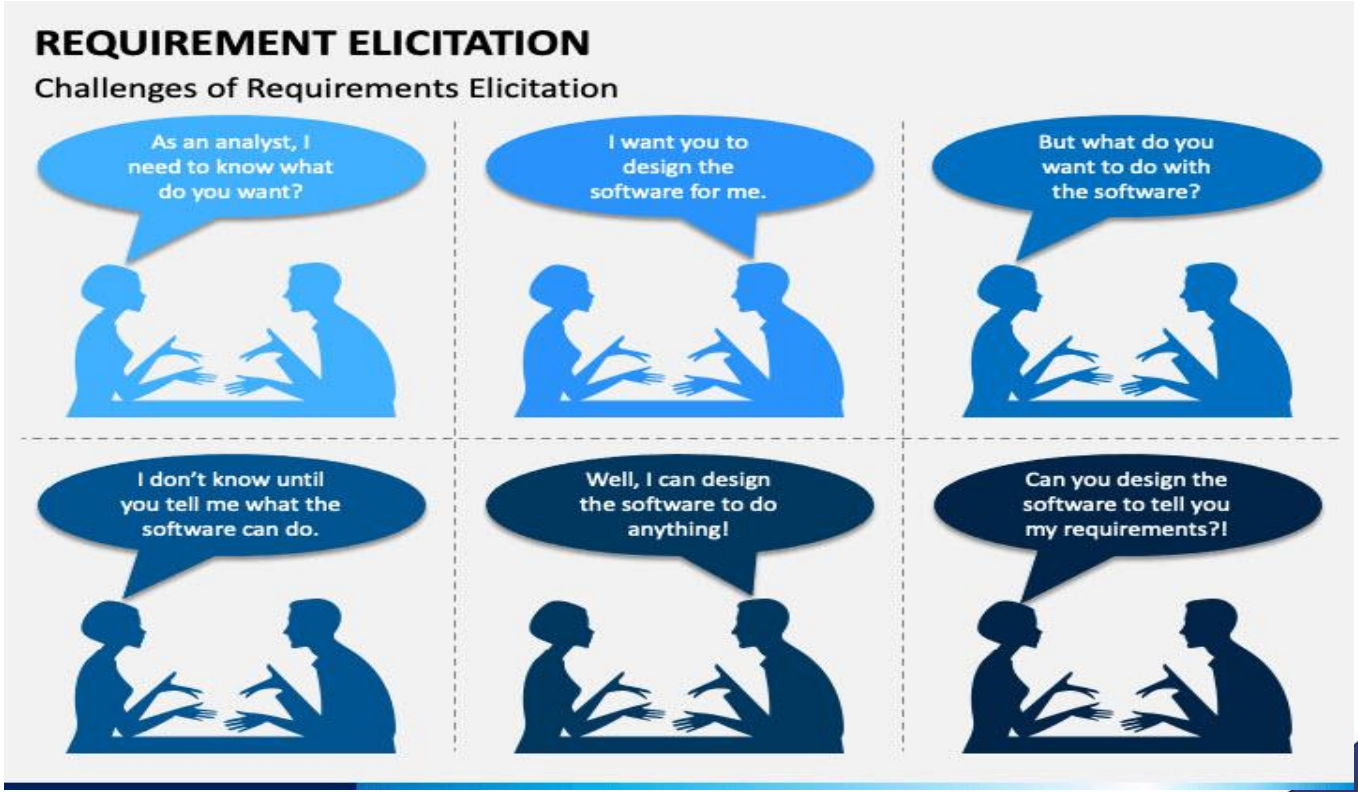
Requirements engineering processes

- Early in the process, most effort will be spent on understanding high-level business and non-functional requirements, and the user requirements for the system.

- Later in the process, in the outer rings of the spiral, more effort will be devoted to eliciting and understanding the non-functional requirements, and more detailed system requirements.



Challenges in eliciting requirements



Requirements elicitation

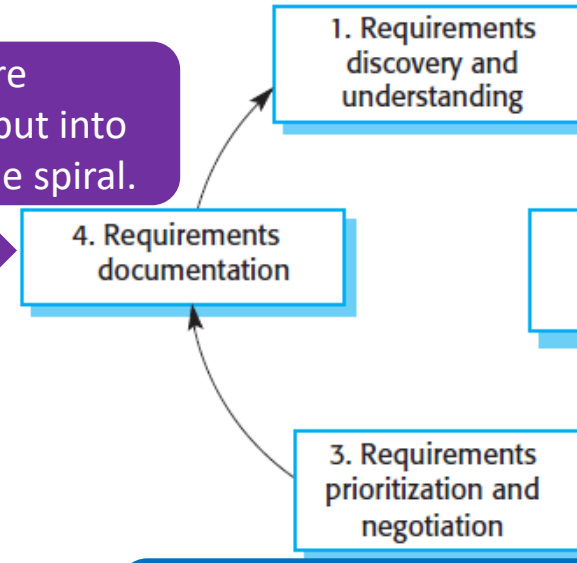
- The aims of the requirements elicitation process are to understand the work that stakeholders do and how they might use a new system to help support their work. Eliciting and understanding requirements from system stakeholders is a difficult process for several reasons:
    - Stakeholders often **don't know what they want** from a computer system except in the most general terms. They may find it difficult to articulate what they want the system to do. They may make unrealistic demands because they don't know what is and what isn't feasible.
    - Different stakeholders, **with diverse requirements**, may **express their requirements in different ways**. Requirements engineers have to discover all potential sources of requirements and discover commonalities and conflicts.

Requirements elicitation

• Eliciting and understanding requirements from system stakeholders is a difficult process for several reasons (cont'd):

- **Political factors** may influence the requirements of a system. Managers may demand specific system requirements because these will allow them to increase their influence in the organisation.
- The **economic and business environment** in which the analysis takes place is dynamic. It inevitably changes during the analysis process. New requirements may emerge. The importance of particular requirements may change.

Process model for elicitation and analysis



This is the process of interacting with stakeholders of the system to discover their requirements.

The requirements are documented and input into the next round of the spiral.

This activity takes the unstructured collection of requirements and organises them into coherent clusters.

This activity is concerned with prioritising requirements and finding and resolving requirements conflicts.

Requirements elicitation techniques

- There are two fundamental approaches to requirements elicitation:
  - **Interviewing**, where you talk to people about what they do.
  - **Observation** or **ethnography**, where you watch people doing their job to see what artifacts they use, how they use them, and so on.

- Formal or informal interviews with system stakeholders are part of most requirements engineering processes. In these interviews, the requirements engineers put questions to stakeholders about the system that they currently use, and the system to be developed.

Requirements elicitation techniques: Interviews

- Interviews may be of two types:
  - **Closed interviews**, where the stakeholder answers a predefined set of questions.
  - **Open interviews**, in which there is no predefined agenda. The requirements engineering team explores a range of issues with system stakeholders and hence develops a better understanding of their needs.

- In practice, interviews with stakeholders are a mixture of both of these. During an interview, you usually have to ask some questions to get started and to keep the interview focused on the system to be developed.

Requirements elicitation techniques: Interviews

• Interviews are good for getting an overall understanding of what stakeholders do, because people like talking about their work, and so they are usually happy to get involved in interviews. However, eliciting domain knowledge through interviews can be difficult, for two reasons:

- All application specialists like to use **jargon** specific to their area of work. They normally use words in a precise and subtle way that requirements engineers may misunderstand.
- Some domain knowledge is so familiar to stakeholders that they find it difficult to explain or they think it is so fundamental that it isn't worth mentioning.
- For example, a librarian knows that all acquisitions must be catalogued before they are added to the library, but this may not be obvious to the interviewer, so it isn't taken into account in the requirements.

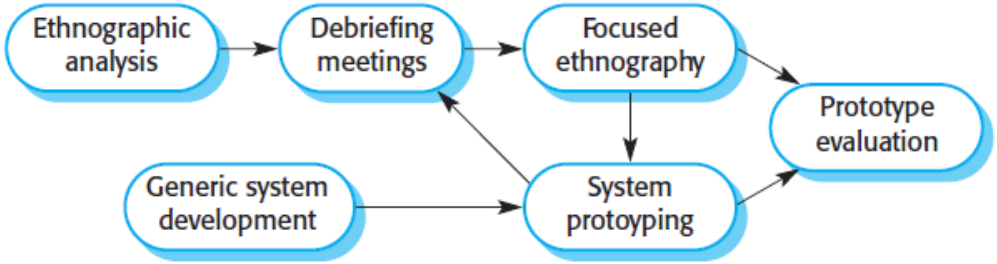
Requirements elicitation techniques: Interviews

- To be an effective requirements interviewer, bear in mind the following:
  - Be open minded, avoid preconceived ideas about the requirements, and be willing to listen to stakeholders.
  - Use springboard questions or a requirements proposal to prompt the interviewee to get discussions going, or by working together on a prototype system. People find it easier to talk in a defined context, rather than in general terms.

- Interviewing on its own is liable to miss essential information. So, it should be used in conjunction with other requirements elicitation techniques.

Requirements elicitation techniques: Ethnography

- Ethnography is an **observational technique** that can be used to understand operational processes and help derive requirements for software to support these processes. Ethnography is particularly effective for discovering two types of requirements:
  - Requirements derived from the way in which people **actually work**, rather than the way in which business process definitions say they **ought to work**.
  - Requirements derived from cooperation and awareness of other people's activities.



Requirements elicitation techniques: Stories

- Stories and scenarios are ways of capturing how users of a system handle particular situations or imagine things that they might do in a new way of working.
- Stories and scenarios are essentially the same thing. They are a description of how the system can be used for some particular task. They describe **what people do**, **what information they use and produce**, and **what systems they may use in this process**. The difference is in the way stories and scenarios are structured and presented.
    - Stories are usually written as narrative text and present a high-level description of system use.
    - Scenarios are usually written with specific information collected such as inputs and outputs.

Requirements elicitation techniques: Story

Jack is a primary school teacher in Ullapool (a village in northern Scotland). He has decided that a class project should be focused on the fishing industry in the area, looking at the history, development, and economic impact of fishing. As part of this project, pupils are asked to gather and share reminiscences from relatives, use newspaper archives, and collect old photographs related to fishing and fishing communities in the area. Pupils use an iLearn wiki to gather together fishing stories and SCRAN (a history resources site) to access newspaper archives and photographs. However, Jack also needs a photo-sharing site because he wants pupils to take and comment on each other's photos and to upload scans of old photographs that they may have in their families.

Jack sends an email to a primary school teachers' group, which he is a member of, to see if anyone can recommend an appropriate system. Two teachers reply, and both suggest that he use KidsTakePics, a photo-sharing site that allows teachers to check and moderate content. As KidsTakePics is not integrated with the iLearn authentication service, he sets up a teacher and a class account. He uses the iLearn setup service to add KidsTakePics to the services seen by the pupils in his class so that when they log in, they can immediately use the system to upload photos from their mobile devices and class computers.

A user story for the iLearn system

Requirements elicitation techniques: Scenario

Initial assumption: A user or a group of users have one or more digital photographs to be uploaded to the picture-sharing site. These photos are saved on either a tablet or a laptop computer. They have successfully logged on to KidsTakePics.

Normal: The user chooses to upload photos and is prompted to select the photos to be uploaded on the computer and to select the project name under which the photos will be stored. Users should also be given the option of inputting keywords that should be associated with each uploaded photo. Uploaded photos are named by creating a conjunction of the user name with the filename of the photo on the local computer.

On completion of the upload, the system automatically sends an email to the project moderator, asking them to check new content, and generates an on-screen message to the user that this checking has been done.

What can go wrong: No moderator is associated with the selected project. An email is automatically generated to the school administrator asking them to nominate a project moderator. Users should be informed of a possible delay in making their photos visible.

Photos with the same name have already been uploaded by the same user. The user should be asked if he or she wishes to re-upload the photos with the same name, rename the photos, or cancel the upload. If users choose to re-upload the photos, the originals are overwritten. If they choose to rename the photos, a new name is automatically generated by adding a number to the existing filename.

Other activities: The moderator may be logged on to the system and may approve photos as they are uploaded.

System state on completion: User is logged on. The selected photos have been uploaded and assigned a status "awaiting moderation." Photos are visible to the moderator and to the user who uploaded them.

Scenario for uploading photos to the iLearn system

Requirements specification

- Requirements specification is the process of writing down the user and system requirements in a requirements document. Ideally, the user and system requirements should be clear, unambiguous, easy to understand, etc., but in practice this is almost impossible to achieve. Stakeholders interpret the requirements in different ways, and there are often inherent conflicts and inconsistencies in the requirements.
- User requirements are almost always written in natural language supplemented by appropriate diagrams and tables in the requirements document.
- System requirements may also be written in natural language, but other notations based on mathematical notations, etc. can also be used.

Requirements specification

Notation	Description
Natural language sentences	The requirements are written using numbered sentences in natural language. Each sentence should express one requirement.
Structured natural language	The requirements are written in natural language on a standard form or template. Each field provides information about an aspect of the requirement.
Graphical notations	Graphical models, supplemented by text annotations, are used to define the functional requirements for the system. UML (unified modeling language) use case and sequence diagrams are commonly used.
Mathematical specifications	These notations are based on mathematical concepts such as finite-state machines or sets. Although these unambiguous specifications can reduce the ambiguity in a requirements document, most customers don't understand a formal specification. They cannot check that it represents what they want, and they are reluctant to accept it as a system contract. (I discuss this approach, in Chapter 10, which covers system dependability.)

Natural language specification

- Natural language has been used to write requirements for software since the 1950s. It is expressive, intuitive, and universal, but it is also potentially vague and ambiguous, and its interpretation depends on the background of the reader. To minimise misunderstanding when writing natural language requirements, the following guidelines can be used:
    - Invent a standard format and ensure all requirement definitions adhere to that format.
    - Use language consistently to distinguish between mandatory and desirable requirements. Mandatory requirements are usually written using ``shall''. Desirable requirements are not essential and are usually written using ``should''.
    - Use text highlighting (e.g., bold, italic, colour) to pick out key parts of the requirement.

Natural language specification

• To minimise misunderstanding when writing natural language requirements, the following guidelines can be used (cont'd):

- Wherever possible, avoid the use of jargon, abbreviations, and acronyms.
- Wherever possible, try to associate a rationale with each user requirement. The rationale explains why the requirement has been included and who proposed the requirement, so that you know whom to consult if the requirement has to be changed.

3.2 The system shall measure the blood sugar and deliver insulin, if required, every 10 minutes. (*Changes in blood sugar are relatively slow, so more frequent measurement is unnecessary; less frequent measurement could lead to unnecessarily high sugar levels.*)

3.6 The system shall run a self-test routine every minute with the conditions to be tested and the associated actions defined in Table 1. (*A self-test routine can discover hardware and software problems and alert the user to the fact the normal operation may be impossible.*)

Structured specification

- Structured natural language is a way of writing system requirements where requirements are written in a standard way rather than as free-form text. This approach maintains most of the expressiveness and understandability of natural language but ensures that some uniformity is imposed on the specification. Structured language notations use templates to specify system requirements, and it may use programming language constructs such as *if-then-else* and *for-1-to-n* to show alternatives and iterations.
- When a standard format is used for specifying functional requirements, the following information should be included:
    - A description of the function or entity being specified.
    - A description of its inputs and the origin of these inputs.

Structured specification

• When a standard format is used for specifying functional requirements, the following information should be included (cont'd):

- A description of its output and the destination of these outputs.
- Information about the information needed for the computation or other entities in the system that are required.
- A description of the action to be taken.
- If a functional approach is used, a *precondition* setting out what must be true before the function is called, and a *postcondition* specifying what is true after the function is called.
- A description of the side effects (if any) of the operation.

Structured specification

Insulin Pump/Control Software/SRS/3.3.2

Function	Compute insulin dose: Safe sugar level.
Description	Computes the dose of insulin to be delivered when the current measured sugar level is in the safe zone between 3 and 7 units.
Inputs	Current sugar reading (r2), the previous two readings (r0 and r1).
Source	Current sugar reading from sensor. Other readings from memory.
Outputs	CompDose—the dose in insulin to be delivered.
Destination	Main control loop.
Action:	CompDose is zero if the sugar level is stable or falling or if the level is increasing but the rate of increase is decreasing. If the level is increasing and the rate of increase is increasing, then CompDose is computed by dividing the difference between the current sugar level and the previous level by 4 and rounding the result. If the result, is rounded to zero then CompDose is set to the minimum dose that can be delivered. (see Figure 4.14)
Requires	Two previous readings so that the rate of change of sugar level can be computed.
Precondition	The insulin reservoir contains at least the maximum allowed single dose of insulin.
Postcondition	r0 is replaced by r1 then r1 is replaced by r2.
Side effects	None.





Structured specification

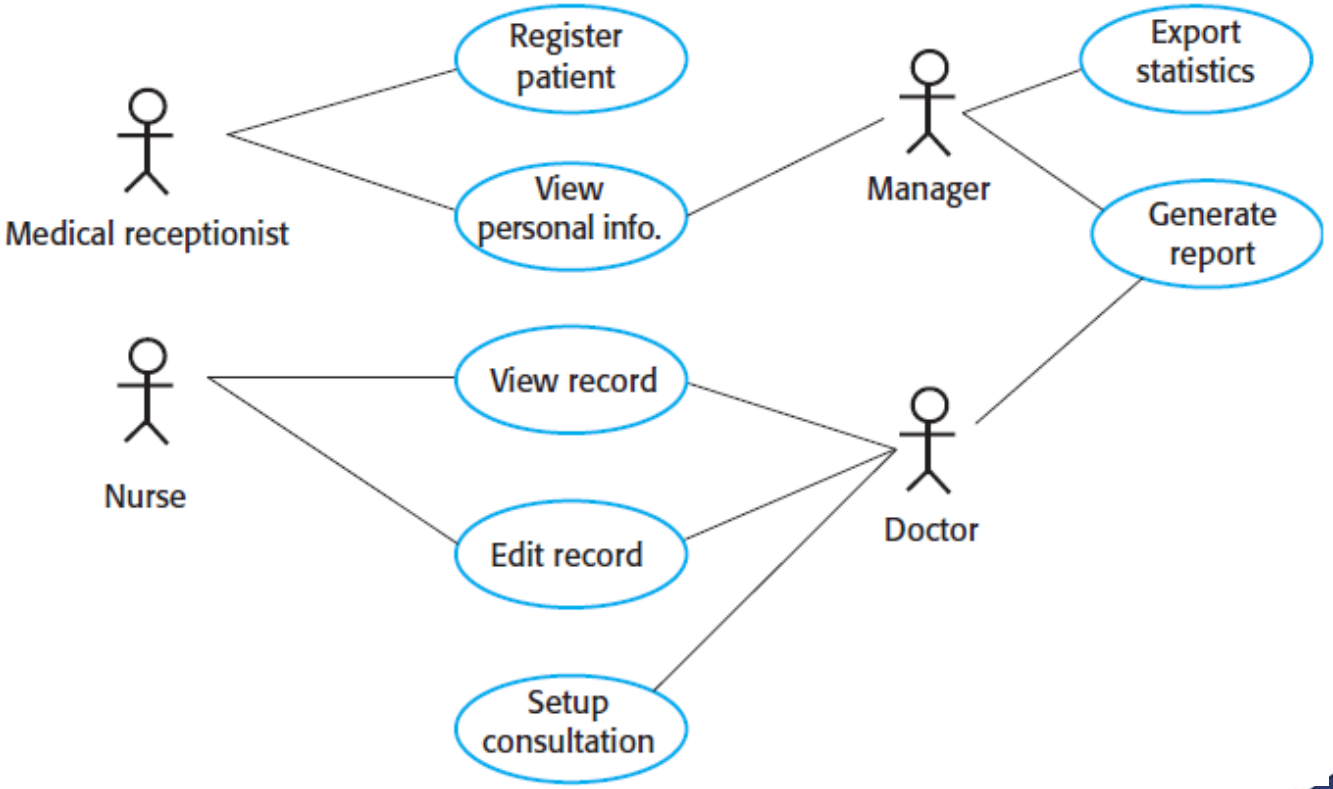
- The structured specifications removes some of the problems of natural language specification. Variability in the specification is reduced, and requirements are organised more effectively. However, it is still difficult to write requirements in a clear and unambiguous way, particularly when complex computations are to be specified. To address this problem, extra information such as tables or graphical models of the system can be added.

Condition	Action
Sugar level falling (r2 < r1)	CompDose = 0
Sugar level stable (r2 = r1)	CompDose = 0
Sugar level increasing and rate of increase decreasing ((r2 − r1)< (r1 − r0))	CompDose = 0
Sugar level increasing and rate of increase stable or increasing r2 > r1 & ((r2 − r1) ≥ (r1 − r0))	CompDose = round ((r2 − r1)/4) If rounded result = 0 then CompDose = MinimumDose

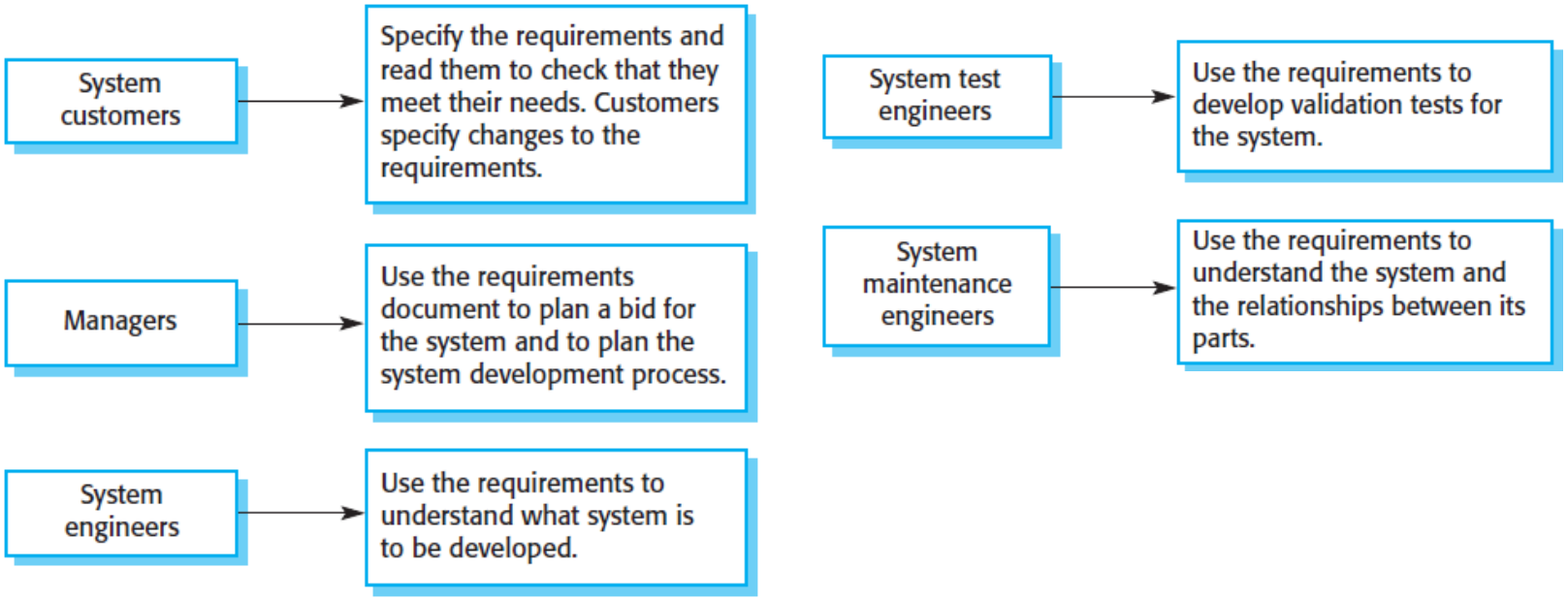
Use cases

- Use cases are a way of describing interactions between users and a system using a graphical model and structured text. Use cases were first introduced in the Objectory method (Jacobsen et. al., 1993) and have now become a fundamental feature of the Unified Modeling Language (UML).
- In their simplest form, a use case identifies the ***actors* involved in an interaction** and **names the type of interaction**.
- Additional information that describes the interaction with the system can be added; it may be a textual description or one or more graphical models such as UML sequence diagrams or state charts.

Use cases: Mentcare system



Users of the software requirements document



Structure of a requirements document

- Based on an IEEE standard for requirements document (IEEE 1998). This standard is a generic one that can be adapted to specific uses.

Chapter	Description
Preface	This defines the expected readership of the document and describe its version history, including a rationale for the creation of a new version and a summary of the changes made in each version.
Introduction	This describes the need for the system. It should briefly describe the system's functions and explain how it will work with other systems. It should also describe how the system fits into the overall business or strategic objectives of the organization commissioning the software.
Glossary	This defines the technical terms used in the document. You should not make assumptions about the experience or expertise of the reader.
User requirements definition	Here, you describe the services provided for the user. The nonfunctional system requirements should also be described in this section. This description may use natural language, diagrams, or other notations that are understandable to customers. Product and process standards that must be followed should be specified.

Structure of a requirements document (cont'd)

System architecture	This chapter presents a high-level overview of the anticipated system architecture, showing the distribution of functions across system modules. Architectural components that are reused should be highlighted.
System requirements specification	This describes the functional and nonfunctional requirements in more detail. If necessary, further detail may also be added to the nonfunctional requirements. Interfaces to other systems may be defined.
System models	This chapter includes graphical system models showing the relationships between the system components and the system and its environment. Examples of possible models are object models, data-flow models, or semantic data models.
System evolution	This describes the fundamental assumptions on which the system is based, and any anticipated changes due to hardware evolution, changing user needs, and so on. This section is useful for system designers as it may help them avoid design decisions that would constrain likely future changes to the system.
Appendices	These provide detailed, specific information that is related to the application being developed—for example, hardware and database descriptions. Hardware requirements define the minimal and optimal configurations for the system. Database requirements define the logical organization of the data used by the system and the relationships between data.
Index	Several indexes to the document may be included. As well as a normal alphabetic index, there may be an index of diagrams, an index of functions, and so on.

Requirements validation

- Requirements validation is the process of checking that requirements define the system that the customer really wants. It overlaps with elicitation and analysis, as it is concerned with finding problems with the requirements.
- Different types of checks should be carried out on the requirements document. These checks include:
    - **Validity checks**. These check that the requirements reflect the real needs of system users.
    - **Consistency checks**. Requirements in the document should not conflict.
    - **Completeness checks**. The requirements document should include requirements that define all functions and the constraints intended by the system user.

Requirements validation

• Different types of checks should be carried out on the requirements document. These checks include (cont'd):

- **Realism checks**. By using knowledge of existing technologies, the requirements should be checked to ensure that they can be implemented within the proposed budget for the system. These checks should also take account of the budget and schedule for the system development.
- **Verifiability**. To reduce potential for dispute between customer and contractor, system requirements should always be written so that they are verifiable. This means that you should be able to write a set of tests that can demonstrate that the delivered system meets each specified requirement.

Requirements validation

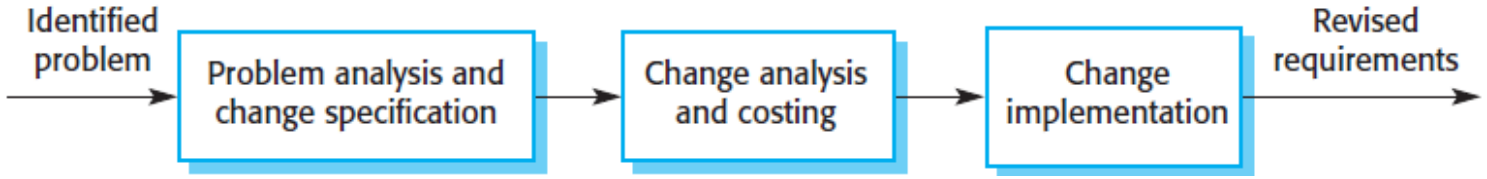
- A number of requirements validation techniques can be used individually or in conjunction with one another:
    - *Requirements reviews*. The requirements are analysed systematically by a team of reviewers who check for errors and inconsistencies.
    - *Prototyping*. This involves developing an executable model of a system and using this with end-users and customers to see if it meets their needs and expectations.
    - *Test-case generation*. Requirements should be testable. If the tests for the requirements are devised as part of the validation process, this often reveals requirements problems. If a test is difficult or impossible to design, this usually means that the requirements will be difficult to implement and should be reconsidered.

Requirements change

- The requirements for large software systems are always changing. Most changes to system requirements arise because of changes to the business environment of the system:
    - The business and technical environment of the system always changes after installation. New hardware may be introduced and existing hardware updated. It may be necessary to interface the system with other systems.
    - The people who pay for a system and the users of that system are rarely the same people. New features may have to be added for user support if the system is to meet its goals.
    - Large system usually have a diverse stakeholder community, with stakeholders having different requirements.

Requirements change management

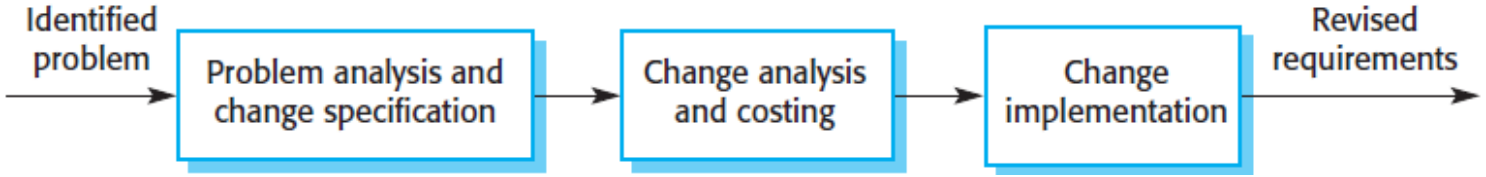
- Requirements management planning is concerned with establishing how a set of evolving requirements will be managed. There are three stages to a change management process:



- **Problem analysis and change specification**. Start with an identified requirements problem, then analyse the change proposal to check if it is valid.
- This analysis is fed back to the change requestor, who may decide to give a more detailed proposal, or withdraw the request.

Requirements change management

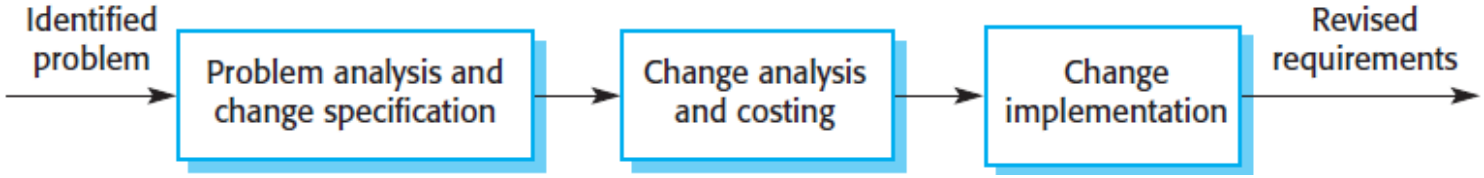
- There are three stages to a change management process (cont'd):



  - **Change analysis and costing**. The effect of the proposed change is assessed using traceability information and general knowledge of the system requirements.
  - The cost is estimated in terms of modifications to the requirements, and to the system design and implementation.
  - Once the analysis is completed, a decision is made as to whether or not to proceed with the change.

Requirements change management

- There are three stages to a change management process (cont'd):



  - **Change implementation**. The requirement document and, where necessary, the system design and implementation, are modified.
  - The requirements document should be organised so that changes can be made to it without extensive rewriting or reorganisation.
  - This can be achieved by making the document as modular as possible and by minimising external references.

Lessons learned

- We defined requirements engineering, user and system requirements and showed examples of user and system requirements.

- We described functional and non-functional requirements, and discussed the differences between functional and non-functional requirements.

- We described the requirements engineering process, and discussed the main activities of elicitation, specification, validation and change in the RE process.